# seL4 Walkthrough

2026-04-19 | UNTIL. LT #0x0A

わたすけ(@watasuke1024)

# わたすけ

## watasuke

- 筑波大学 mast 4 年
- **Twitter: watasuke1024**
  その他: watasuke102
- Rust で自作 OS をやっていた（やりたい）

# 目次

1. seL4？
2. seL4 の概念
3. スレッドを起動してみる

# 目次

1. **seL4？**
2. seL4 の概念
3. スレッドを起動してみる

# OS のカーネルって 200 色ある

- Linux カーネルなどは**モノリシックカーネル**と呼ばれる
  - I/O やデバイスといった様々な機能を
    カーネル空間で実装
- 対義語は**マイクロカーネル**
  - カーネルが**最小限の機能を持つ**
  - 「モノリシックカーネル」という語は
    これが登場してから生まれたらしい（レトロニム）

# マイクロカーネル、よさげ

- モノリシックカーネルが持っていた機能は、ユーザーランドで**サーバー**として実装
  - OS が提供する機能を変えやすい
- カーネルが小さいから attack surface が小さい！デバッグが楽！

# マイクロカーネル、渋い

## 遅くね？

- サーバー同士のやりとりは
  **IPC** (Inter-Process Communication) で行う
  - 情報処理推進特別委員会ではないですよ！
- ボトルネックになりがち
  - IPC 用のシステムコール、コンテキストスイッチ、etc…

# マイクロカーネル、渋くない！

- IPC←ちゃんと作ると速くなる（575）
- 高速な IPC を特徴とする **L4** の登場
- Apple のセキュリティチップ「Secure Enclave」で L4 が動いている
  - iPhone や MacBook に搭載されている（いた?）

# seL4

- secure L4
- **仕様が Haskell で書かれている**
  - そこから C のコードを自動生成
- Isabelle/HOL による**形式検証が行われている**

# 目次

1. seL4？
2. **seL4 の概念**
3. スレッドを起動してみる

# Capability (ケーパビリティ)

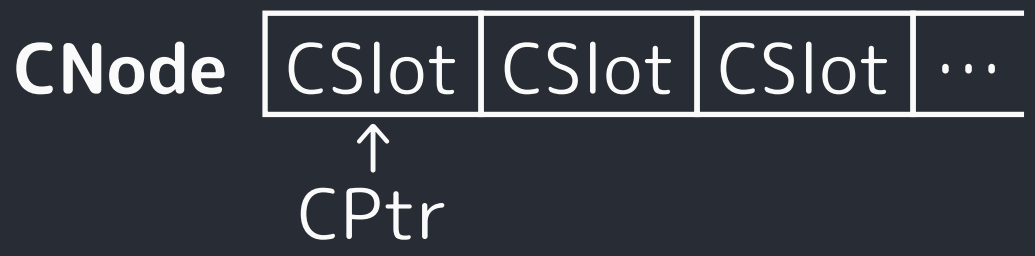
- 「何にアクセスできるか」を示すもの
- 3 種類ある：
  1. **カーネルオブジェクト**へのアクセスを制御
  2. **抽象リソース**へのアクセスを制御
  3. **Untyped メモリ**へのアクセスを制御

```haskell
data Capability
        = ThreadCap {
            capTCBPtr :: PPtr TCB }
        | NullCap
        | ReplyCap {
            capTCBPtr :: PPtr TCB,
            capReplyMaster :: Bool,
            capReplyCanGrant :: Bool }
        | IRQControlCap
        deriving (Eq, Show)
```

Haskell による Capability の定義（一部）

# CNode と Capability

- Capability は **CNode** という配列に格納される
- CNode の各コンテナを **CSlot** と呼ぶ
- 位置の指定には **CPtr** を用いる
  - メモリアドレスを指すポインタではない



## Capability

- カーネルが起動してから最初に起動する**ルートタスク**が全ての Capability を持っている
- ルートタスクが自身の Capability をコピーして他のタスクに分け与える

```
seL4_CNode_Copy(
  // dest
  seL4_CapInitThreadCNode, boot_info->empty.start, seL4_WordBits,
  // src
  seL4_CapInitThreadCNode, seL4_CapInitThreadTCB,  seL4_WordBits,
  seL4_AllRights
);
```

ルートタスクが持つ CNode の TCB Capability を
空きスロットにコピーする例

## Untyped memory

- 連続した物理メモリの領域
- これのリストが、それを操作する Capability とともに
  ルートタスクに渡される
- Untyped Capability を、**カーネルオブジェクト**と
  それを操作する Capability に **Retype** できる

```
seL4_Untyped_Retype(
  parent_untyped, // 変更元の Untyped Memory
  seL4_TCBObject, // 作成したいオブジェクトの種類
  seL4_TCBBits,   // オブジェクトのサイズ
  // 作成したCapabilityを格納するCSlot
  seL4_CapInitThreadCNode, // root
  0,                       // node_index
  0,                       // node_depth
  tcb_slot,                // node_offset
  1                        // 作成個数
);
```

```
seL4_CPtr tcb_slot = ...;
seL4_Untyped_Retype(seL4_TCBObject, ...,
    tcb_slot, 1);
// System Call!
seL4_TCB_Configure(tcb_slot, ...);
```

# 目次

1. seL4？
2. seL4 の概念
3. **スレッドを起動してみる**

# スレッドを作る方法

- カーネルオブジェクトである
  **TCB (Thread Control Block)** を作成
  - これを作るにはもちろん Capability が必要
- TCB が持つレジスタの値を書き換える
  - **rip** (Instruction Pointer) や **rsp** (Stack Pointer)

## やってみよう！

- 本当は ELF をロードしたかった
- 時間が足りませんでした泣
- メモリに機械語を直接書き込む脳筋スタイルで行く

# やってみた

```
seL4_CPtr create_object(seL4_BootInfo *boot_info, seL4_Word type) {
  seL4_CPtr cslot = boot_info->empty.start++;
  for (seL4_CPtr i = boot_info->untyped.start;
       i < boot_info->untyped.end; ++i) {
    if (boot_info->untypedList[i - boot_info->untyped.start].isDevice) {
      continue;
    }
    seL4_Error result = seL4_Untyped_Retype(i, type, 0,
      seL4_CapInitThreadCNode, 0, 0, cslot, 1);
    if (result == seL4_NoError) return cslot;
  }
}
```

# やってみた

```
seL4_CPtr create_object(seL4_BootInfo *boot_info, seL4_Word type) {
  seL4_CPtr cslot = boot_info->empty.start++;
  for (seL4_CPtr i = boot_info->untyped.start;
       i < boot_info->untyped.end; ++i) {
    if (boot_info->untypedList[i - boot_info->untyped.start].isDevice) {
      continue;
    }
    seL4_Error result = seL4_Untyped_Retype(i, type, 0,
      seL4_CapInitThreadCNode, 0, 0, cslot, 1);
    if (result == seL4_NoError) return cslot;
  }
}
```

# メモリのマッピング

```
const seL4_Word kMapAddr = 0xA000000000;
seL4_CPtr frame = create_object(boot_info, seL4_X86_4K);
seL4_CPtr pdpt  = create_object(boot_info, seL4_X86_PDPTObject);
seL4_CPtr pd    = create_object(boot_info, seL4_X86_PageDirectoryObject);
seL4_CPtr pt    = create_object(boot_info, seL4_X86_PageTableObject);
seL4_X86_PDPT_Map(pdpt, seL4_CapInitThreadVSpace, kMapAddr,
                  seL4_X86_Default_VMAttributes);
seL4_X86_PageDirectory_Map(pd, seL4_CapInitThreadVSpace, kMapAddr,
                           seL4_X86_Default_VMAttributes);
seL4_X86_PageTable_Map(pt, seL4_CapInitThreadVSpace, kMapAddr,
                       seL4_X86_Default_VMAttributes);
seL4_X86_Page_Map(frame, seL4_CapInitThreadVSpace, kMapAddr,
                  seL4_ReadWrite, seL4_X86_Default_VMAttributes);
```

# コンパイラいつもありがとう

```
$ bat a.c
int main(int argc, char** argv) {
  *((volatile uint32_t*)0xA000000300) += 1;
  return 0;
}
$ clang -O0 -g -c a.c
```

# binutils いつもありがとう

```
$ objdump -dfS a.o
  48 b8 00 03 00 00 a0  movabs rax,0xa000000300
  00 00 00
  ff 00                 inc    DWORD PTR [rax]
```

わかりやすくするために出力結果を切り貼りしています

# 機械語かきまくり

```
uint8_t *p = (void *)kMapAddr;
memset(p, 0, 1024 * 4);
// movabs rax, 0xA0'0000'0300
p[0] = 0x48; p[1] = 0xb8; p[3] = 0x03; p[6] = 0xa0;
// inc [rax]
p[10] = 0xff; p[11] = 0x00;
// jmp -3
p[12] = 0xeb; p[13] = 0xfc;
```

# TCB の作成

```
seL4_CPtr tcb = create_object(boot_info, seL4_TCBObject);
seL4_TCB_Configure(tcb,
  seL4_CapNull,                    // fault handler
  seL4_CapInitThreadCNode, 0,  // CNode
  seL4_CapInitThreadVSpace, 0, // VSpace
  0, 0);
seL4_TCB_SetPriority(tcb, seL4_CapInitThreadTCB, 254);
```

# TCB のレジスタ書き換え

```
seL4_TCB_ReadRegisters(tcb, 0, 0,
  sizeof(regs) / sizeof(seL4_Word), &regs);

regs.rip = kMapAddr;
regs.rsp = &tcb_stack_base + sizeof(tcb_stack_base);

seL4_TCB_WriteRegisters(tcb, 0, 0,
  sizeof(regs) / sizeof(seL4_Word), &regs);
```

# 起動！

```
seL4_TCB_Resume(tcb);

volatile uint32_t *write_dest = (void *)0xA000000300;
while (1) {
  if ((*write_dest / 10) == 1234567) {
    printf("count reached (%u)\n", *write_dest);
    seL4_DebugDumpScheduler();
  }
}
```

# デモ

# うまく動きませんでした

- なんで？
- スレッドが restart のまま動かない（575）
- チュートリアルと違って CapDL とかいう
  謎のやつを使ってないから？